

Yrless

Wireless Security Report - WiFi Performance

Antonio Amendolara
Matricola 364109

Davide Marin
Matricola 360040

Alberto Mercurelli
Matricola 360214

Emanuele Tocci
Matricola 363290

1 INTRODUCTION

In this report, we evaluate the **performance** and **goodput** of **TCP** and **UDP** transmissions across various network media. While theoretical protocol differences are well known, real-world efficiency heavily depends on the **underlying physical layer**. For instance, *Wi-Fi interferences* and channel constraints can trigger TCP error recovery mechanisms or cause UDP packet loss, drastically altering performance compared to stable wired connections.

The **main objective** of this laboratory is to quantify how these external factors and **network bottlenecks** impact overall transmission efficiency. To achieve this, we conducted bidirectional measurements (*Server-to-Client* and *Client-to-Server*) between two personal computers. We leveraged custom automation scripts for reliable data collection and utilized *Wireshark* to plot throughput and I/O graphs for detailed post-analysis.

2 TOOLS AND THEORY

Traffic generation and monitoring were performed using standard networking tools, orchestrated by custom automation scripts to ensure reliability and statistical consistency throughout the evaluation:

- **iperf [9]**: Utilized for the active measurement of the maximum achievable TCP and UDP bandwidth across the network links.
- **Wireshark [10]**: Deployed to capture packet-level traces and generate statistical visualizations, including **I/O graphs** for real-time throughput and **TCP Stream graphs** to examine sequence number evolution, round-trip times, and protocol behavior.
- **Automation Scripts**: A custom Python [8] client-server suite was developed to orchestrate the *iperf* execution, supporting both forward and reverse (-R) modes. To mitigate transient anomalies, the scripts executed **10 sequential tests** with 1-second reporting intervals, automatically parsing the output via regular expressions to log raw data and compute key stability metrics (mean, min, max, and standard deviation).

2.1 Goodput $\hat{G}$ Estimation

Goodput represents the **rate** at which useful data is successfully delivered to the application layer, excluding protocol headers, overhead, and retransmitted packets. To properly evaluate the experimental setup, the goodput $\hat{G}(L)$, expressed as a function of the

application buffer length L , is estimated and compared against its theoretical upper bound. Such bound is obtained from the measured nominal capacity C and the protocol efficiency η_{protocol} through the relation:

$$\hat{G} < \eta_{\text{protocol}} \cdot C. \quad (1)$$

3 LAB SETUP AND SCENARIOS

The evaluation was conducted using two Linux-based hosts and a Wi-Fi 6 Router. *The comprehensive overview of the hardware specifications, exact IP addressing, and interface configurations are detailed in Appendix C.*

Table 1: Testbed Configuration

Parameter	STA 1 - server	STA 2 - client
Hardware OS	Prestige 15 A12UC Kubuntu [7]	ROG Zephyrus G16 CachyOS [2] (Arch)
Wireless Int.	Intel Wi-Fi AX211	Arrow Lake CNVi Wi-Fi
Kernel Ver.	6.12.74-deb13	6.18.22-1-cachyos-lts

3.1 Scenario 1: Both Ethernet

In this scenario, a **direct point-to-point link** was established by connecting the two machines with a single Ethernet cable, completely bypassing any intermediate networking devices such as routers or switches. Because the direct connection lacked a **DHCP server**, the operating systems automatically assigned Link-Local addresses. Since the communication was strictly confined to this direct link within the same local subnet, no default gateway routing was required or utilized.

Regarding the measurement parameters used for this specific configuration, the UDP tests were conducted using an initial, **legacy** version of the automation script A. This specific script version lacked the `-b` bandwidth override flag for the *iperf2* application. As a direct consequence of this omission, *iperf2* restricted the **UDP** traffic generation to its default **cap** of 1 Mbps.

Prior to initiating the traffic generation, the physical link parameters were inspected using the *ethtool* utility to identify potential hardware bottlenecks. The inspection revealed that the server's network interface adapter (`enx7cc2c63eb602`) auto-negotiated a physical link **capacity** of 100 Mbps in *Full-Duplex mode (100baseT/Full)* with the client. Consequently, this represents a strict physical **bottleneck** for the entire end-to-end connection, defining the absolute **upper bound** for the expected goodput.

3.1.1 Theoretical Upper Bounds. To estimate the maximum achievable goodput, the theoretical protocol efficiency (η) was first calculated based on standard protocol overheads over a standard 1500-byte MTU, and subsequently applied to the general goodput formula discussed in Section 2.1.

For **TCP** transmissions, considering a Maximum Segment Size (MSS) of 1460B, a 20B TCP header, a 20B IP header and a 38B Ethernet frame overhead, the theoretical efficiency is:

$$\eta_{TCP} = \frac{1460}{(1460 + 20 + 20) + 38} = 0.949 \quad (2)$$

Yielding an expected maximum TCP Goodput of:

$$\hat{G}_{TCP}^{ETH} \approx 0.949 \cdot 100 \text{ Mbps} = 94.9 \text{ Mbps} \quad (3)$$

For **UDP** transmissions, although the maximum theoretical payload is 1472 B (due to the smaller 8 B transport header), the `iperf2` tool utilizes a default payload of 1470 B. Consequently, the efficiency is calculated as follows:

$$\eta_{UDP} = \frac{1470}{(1470 + 8 + 20) + 38} = 0.957 \quad (4)$$

Resulting in an **expected maximum UDP Goodput** of:

$$\hat{G}_{UDP}^{ETH} \approx 0.957 \cdot 100 \text{ Mbps} = 95.7 \text{ Mbps} \quad (5)$$

3.2 Scenario 2: Both Wireless

In this configuration, both the client (STA_1) and the server (STA_2) are associated with the same wireless access point, a **TIM Hub+** router.

Because IEEE 802.11 operates as a **half-duplex shared medium**, the Access Point cannot transmit and receive simultaneously. Consequently, data delivery requires a mandatory **two-hop transmission process** (Sender $\rightarrow$ AP $\rightarrow$ Receiver). This effectively halves the available airtime, applying a multiplier of 0.5. For modern deployments using IEEE 802.11ax, we assume a baseline protocol efficiency (η) ranging between 0.75 and 0.80.

3.2.1 Theoretical Upper Bounds (Forward & Reverse). Regardless of the transmission direction, the bottleneck capacity (C) is strictly limited by the 1201.0 Mbps interface of STA_1 :

- **Forward** (Arch $\rightarrow$ Kubuntu): $C = \min(TX_{Arch}, RX_{Kubuntu}) = \min(2593.5, 1201.0) = 1201.0 \text{ Mbps}$.
- **Reverse** (Kubuntu $\rightarrow$ Arch): $C = \min(TX_{Kubuntu}, RX_{Arch}) = \min(1201.0, 1921.5) = 1201.0 \text{ Mbps}$.

Applying the shared medium penalty (0.5) and the respective protocol efficiency (η), the **theoretical goodput expectations** $\hat{G}$ for both paths are identical:

$$\hat{G}_{UDP}^{WiFi} < C \cdot \eta_{UDP} \cdot 0.5 = 1201 \cdot 0.80 \cdot 0.5 = 480.40 \text{ Mbps} \quad (6)$$

$$\hat{G}_{TCP}^{WiFi} < C \cdot \eta_{TCP} \cdot 0.5 = 1201 \cdot 0.75 \cdot 0.5 = 450.38 \text{ Mbps} \quad (7)$$

3.3 Scenario 3: Mixed Mode

In this configuration, the communication occurs in a heterogeneous environment. The client (Arch) is connected to the **TIM Hub+** router via a wired Gigabit Ethernet link, while the server (Kubuntu) connects to the same access point via IEEE 802.11ax Wi-Fi.

The fundamental difference from the purely wireless scenario is the **removal of the shared medium** between the two endpoints. Since the AP receives data via the wired backbone and transmits it over the air interface (or vice versa), the mandatory **two-hop transmission is eliminated**.

The overall network efficiency is constrained by the router's RJ45 Ethernet ports, which are limited to 1 Gbps.

3.3.1 Theoretical Upper Bounds. To determine the overall performance, we first evaluate the theoretical limits for the Wi-Fi and Ethernet links independently. For the **UDP** protocol, the expected bounds for each individual link are:

$$\begin{cases} \hat{G}_{UDP}^{Mixed}^{Wifi} < 1201 \text{ Mbps} \cdot 0.80 = 960.80 \text{ Mbps} \\ \hat{G}_{UDP}^{Mixed}^{ETH} < 1000 \text{ Mbps} \cdot 0.957 = 957.00 \text{ Mbps} \end{cases} \quad (8)$$

The maximum sustainable UDP goodput for the entire mixed path is determined by the **slower link** (Ethernet):

$$\hat{G}_{UDP}^{Mixed} < \min\left(\hat{G}_{UDP}^{Mixed}^{Wifi}, \hat{G}_{UDP}^{Mixed}^{ETH}\right) = 957.00 \text{ Mbps} \quad (9)$$

Similarly, for the **TCP** protocol, we calculate the bounds based on the respective protocol efficiencies:

$$\begin{cases} \hat{G}_{TCP}^{Mixed}^{Wifi} < 1201 \text{ Mbps} \cdot 0.75 = 900.75 \text{ Mbps} \\ \hat{G}_{TCP}^{Mixed}^{ETH} < 1000 \text{ Mbps} \cdot 0.949 = 949.00 \text{ Mbps} \end{cases} \quad (10)$$

In this case, the overall TCP goodput is restricted by the wireless segment, yielding:

$$\hat{G}_{TCP}^{Mixed} < \min\left(\hat{G}_{TCP}^{Mixed}^{Wifi}, \hat{G}_{TCP}^{Mixed}^{ETH}\right) = 900.75 \text{ Mbps} \quad (11)$$

4 RESULTS

4.1 TCP Performance

Table 2: Summary of TCP goodput experiments

Test		TCP: Goodput per flow				
		Prediction	Average	Min	Max	Std
Both Ethernet	A	94.9	93.82	93.7	93.9	0.06
	B*	94.9	94.1	94.1	94.1	0.0
Both WiFi	C	450.38	413.80	352.00	465.00	39.40
	D*	450.38	248.26	47.50	407.00	118.81
Mixed mode	E	900.75	732.40	640.00	797.00	44.52
	F*	900.75	798.20	685.00	922.00	64.36

* Note: Tests B, D, and F denote measurements performed in reverse mode (Server $\rightarrow$ Client).

4.1.1 Ethernet Scenario. As summarized in Table 2, the empirical measurements for the direct Ethernet-to-Ethernet scenario align closely with the **theoretical predictions**. While the expected maximum TCP goodput was estimated at 94.9 Mbps, the experimental data yielded an **average goodput** of 93.82 Mbps. This negligible discrepancy can be attributed to minor OS-level processing overheads and interrupts. Furthermore, the extremely low standard deviation of 0.06 Mbps demonstrates the **high stability** and reliability of

the dedicated point-to-point wired link. This outcome successfully establishes a robust performance baseline, completely free from the medium contention and interference typically observed in wireless environments.

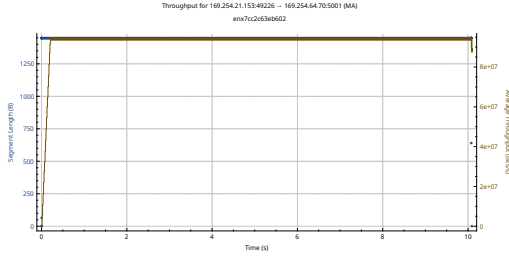


Figure 1: TCP Throughput and Segment Length evolution over time

The graphical representation of the throughput over time, shown in Figure 1, visually confirms the statistical results. Throughput instantly reaches the maximum sustainable plateau without oscillations, while the TCP segment size remains stably fixed at the Maximum Segment Size (MSS) for the entire duration of the transfer, proving the total absence of fragmentation, packet losses or retransmissions.

To evaluate the symmetry of the point-to-point connection, measurements were repeated using the -R (reverse) flag. As expected for a Full-Duplex 100baseT Ethernet link, the results were perfectly symmetrical, yielding an average goodput of 93.75 Mbps.

4.1.2 WiFi Scenario. In the current Wi-Fi scenario, the **theoretical upper bound** for the TCP goodput was established at 450.38 Mbps (see Section 3.2).

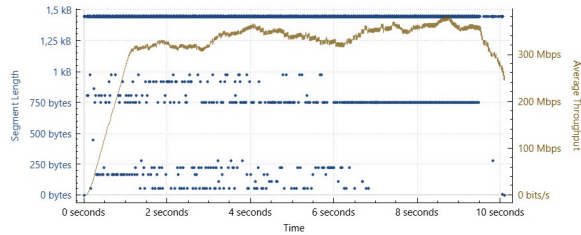


Figure 2: TCP Throughput

However, empirical measurements recorded an average real-world goodput G between 413.80 Mbps and 232.00 Mbps. This **performance degradation** can be analyzed through the following observations:

- **Throughput Variability:** As shown in Figure 2, the instantaneous throughput exhibits **high variance** and **frequent oscillations**. These fluctuations are the direct result of the CSMA/CA contention process, where the available airtime is dynamically shared between the two wireless hops and the background signaling of the 5200 MHz band.
- **Protocol Stalls:** The Stevens graph (Figure 4) displays a linear progression of sequence numbers interrupted by **occasional horizontal segments**. These plateaus represent **transmission stalls** caused by physical layer **frame loss**. During these intervals, the TCP sender is forced to wait for retransmissions or timeout expiration, leading to a temporary **reduction in the effective goodput**.

- **I/O Correlation:** The I/O graph (Figure 3) confirms the throughput patterns, showing a packet rate that fluctuates in sync with the channel availability.

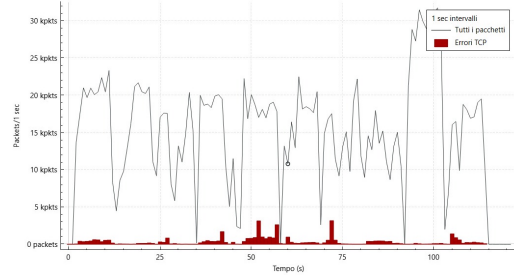


Figure 3: Wireshark I/O graph representing the packet transfer rate over time for the forward TCP flow. The continuous fluctuations in the packet rate correlate with MAC-layer channel contention, validating the instability observed in the throughput metrics.

Ultimately, the significant gap between the predicted upper bound ($\hat{G}_{WiFi}^{UDP} = 450.38$ Mbps) and the empirical average (232.00 Mbps) confirms that environmental noise, AP queuing delays, and MAC-layer overhead are the **primary bottlenecks** preventing the flow from sustaining its theoretical maximum capacity.

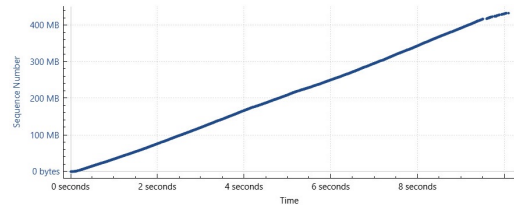


Figure 4: Stevens TCP sequence number graph highlighting temporary transmission stalls in the Wireless scenario

4.1.3 Mixed Scenario. The empirical data demonstrates a substantial performance **improvement** compared to the purely **wireless setup**. By removing the mandatory *two-hop* wireless transmission, the **TCP goodput approaches** the upper bounds dictated by the 1 Gbps Ethernet interface and the 802.11ax protocol efficiency.

- **Throughput Gains and Physical Limits:** As shown in the Throughput graph (Figure 5), the transfer rate ramps up quickly and sustains values between 800 and 900 Mbps. This aligns perfectly with the **average goodput measurements** (732.40 Mbps in forward mode and 798.20 Mbps in reverse mode). The highest peaks demonstrate that under optimal radio conditions, the wireless segment pushes data fast enough to **saturate** the physical 1 Gbps port of the router.
- **Protocol Stalls:** Despite the high overall data rate, the connection still experiences residual **channel instability**. This instability manifests as temporary **transmission stalls**. During these brief fractions of a second, the TCP sender is forced to pause and wait for retransmissions due to **frame loss** on the Wi-Fi hop.
- **TCP Errors and Contention:** Continuous fluctuations in the overall packet rate, combined with the occurrence of explicit TCP errors, confirm that MAC-layer contention and

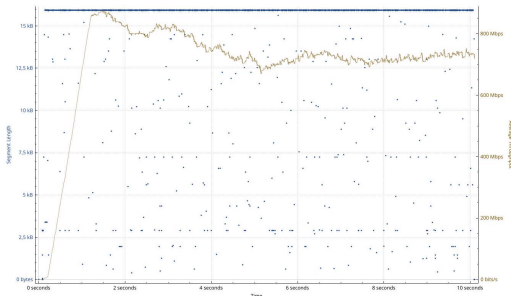


Figure 5: TCP average throughput and segment length evolution over time in the Mixed scenario

environmental noise continue to impact the flow. These underlying packet-level errors are the direct trigger for the aforementioned protocol stalls, which in turn cause momentary throughput dips.

4.2 UDP Performance

Table 3: Summary of UDP goodput experiments

Test		UDP: Goodput per flow				
		Prediction	Average	Min	Max	Std
Both Ethernet	A	97.5	1.05	1.05	1.05	0.00
	B*	97.5	1.05	1.05	1.05	0.00
Both WiFi	C	480.40	318.30	268.00	377.00	33.75
	D*	480.40	238.50	231.00	246.00	10.61
Mixed mode	E	957.00	777.90	524.00	905.00	124.96
	F*	957.00	283.43	277.00	288.00	3.82

* Note: Tests B, D, and F denote measurements performed in reverse mode (Server → Client).

4.2.1 Ethernet Scenario. During the empirical evaluation of the UDP protocol over the direct 100baseT link, the data revealed a consistent, flat bandwidth of exactly 1.05 Mbps across all 10 iterations. This discrepancy from the theoretical optimum is not a hardware bottleneck, but a direct consequence of the iperf application’s **default behavior**, which restricts UDP traffic generation to 1 Mbps unless explicitly overridden by the -b flag. Despite this limitation in throughput probing, the server reports provide highly valuable insights into the channel quality. Across all iterations, the server recorded exactly *0 lost datagrams* (0% packet loss) and a Jitter of 0.000 ms. This absolute **reliability** corroborates the exceptional stability of the dedicated wired link at low transmission rates. Furthermore, the subsequent high-throughput results observed in the Mixed Scenario definitively confirm that the underlying Ethernet medium was capable of sustaining near-theoretical UDP loads when properly configured.

4.2.2 WiFi Scenario. The evaluation of UDP performance highlights the wireless medium’s raw capacity in the absence of transport-layer congestion control. Despite a **theoretical upper bound** of 480.40 Mbps, **empirical results** (Table 3) fall significantly **lower**. Because UDP does not scale back transmission rates during packet loss, it rapidly **saturates the channel**, leaving performance entirely bottlenecked by CSMA/CA contention. Additionally, a high

standard deviation of 33.75 Mbps demonstrates significant vulnerability to transient **environmental noise**.

4.2.3 Mixed Scenario. In the mixed topology (Ethernet-to-Wi-Fi), UDP highlights the raw transmission capacity of the hybrid link by bypassing transport-layer congestion control and ignoring RTT variations. Pushing data aggressively, the **average goodput** reaches 674.9 Mbps, significantly improving upon the purely wireless setup and marginally outperforming TCP. However, performance still falls short of the 957 Mbps theoretical bound. The Wi-Fi segment remains the primary **bottleneck**: a standard deviation of 28 Mbps confirms that protocol-agnostic physical constraints ultimately dictate the overall network stability.

5 CONCLUSIONS

This experiment evaluated the **performance** of TCP and UDP transmission protocols across different network topologies to better understand how physical conditions impact throughput quality. As expected, the baseline Ethernet-to-Ethernet connection significantly outperformed the Wi-Fi setups. The wired medium introduced minimal overhead, allowing both protocols to achieve a goodput *very close to the theoretical optimum*. Specifically, TCP performance was marginally lower than the absolute maximum but remained remarkably close to the physical limit of the Ethernet link.

Conversely, the pure **wireless scenario** (Wi-Fi-to-Wi-Fi) exhibited the **worst results**. Due to the inherent overhead of the 802.11 MAC layer and environmental interference, the measured TCP goodput deteriorated to an average between 232.00 Mbps and 413.80 Mbps, falling significantly short of the estimated 450.38 Mbps theoretical capacity. In both setups, the UDP measurements hit a strict 1 Mbit/s ceiling; however, as previously noted, this was due to the default maximum transmission rate of the iperf testing tool rather than a physical or protocol bottleneck.

Ultimately, the experiment demonstrated that **Ethernet connections** provide a vastly superior and **more stable** goodput compared to Wi-Fi, which is heavily constrained by environmental *interference* and protocol overhead. Comparing the transport layers, TCP proved its reliability; its error recovery and congestion control mechanisms make it highly effective at mitigating the transmission problems inherent to wireless environments.

REFERENCES

- [1] Arch Linux Development Team. 2026. Arch Linux: A simple, lightweight Linux distribution. (2026). https://archlinux.org/
- [2] CachyOS Team. 2026. CachyOS: An Arch Linux-based distribution focused on performance and speed. (2026). https://cachyos.org/
- [3] Canonical Ltd. 2023. Ubuntu: An open-source software platform that runs everywhere from the PC to the server and the cloud. (2023). https://ubuntu.com/
- [4] IEEE. 2018. IEEE Standard for Ethernet. (2018). https://standards.ieee.org/standard/802_3-2018.html
- [5] IEEE. 2021. IEEE Standard for Information technology–Telecommunications and information exchange between systems Local and metropolitan area networks–Specific requirements Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications Amendment 5: Enhancements for High Efficiency WLAN. (2021). https://standards.ieee.org/standard/802_11ax-2021.html
- [6] Jangeun Jun, P. Peddabachagari, and M. Sichitiu. 2003. Theoretical maximum throughput of IEEE 802.11 and its applications. In *Second IEEE International Symposium on Network Computing and Applications, 2003. NCA 2003*. 249–256. <https://doi.org/10.1109/NCA.2003.1201163>
- [7] Kubuntu Community. 2026. Kubuntu: Friendly Computing based on KDE Plasma and Ubuntu. (2026). https://kubuntu.org/
- [8] Python Software Foundation. 2023. Python: A programming language that lets you work quickly and integrate systems more effectively. (2023). https://www.python.org/
- [9] The iperf2 Development Team. 2023. iperf2: A tool for measuring TCP and UDP bandwidth performance. (2023). https://sourceforge.net/projects/iperf2/
- [10] The Wireshark Foundation. 2023. Wireshark: The world's foremost network protocol analyzer. (2023). https://www.wireshark.org/

A MEASUREMENT SCRIPT

The following Python script automates the network performance evaluation using iperf, as described in Section 2. It represents the final, optimized version used for the Wireless and Mixed scenarios. It includes the `-b 1000M` flag to unlock UDP bandwidth. The preliminary Ethernet UDP tests were conducted using an earlier iteration of this script lacking this specific flag.

```
import subprocess, re, statistics, time, sys

def main():
    print("=== iperf Client Mode ===")
    ip = input("Server IP: ").strip()
    proto = input("Protocol (tcp/udp): ").strip().lower()
    method = input("Mode (standard/reverse): ").strip().lower()

    cmd = (
        ["iperf", "-c", ip, "-i", "1"]
        + (["-u", "-b", "1000M"] if proto == "udp" else [])
        + (["-R"] if method == "reverse" else [])
    )
    log_file, tputs = f"log_{proto}_{ip}.txt", []
    print(f"\nGenerated iperf command: {' '.join(cmd)}")
    print(f"Running 10 tests... Logs will be saved to {log_file}\n")

    for i in range(1, 11):
        try:
            out = subprocess.run(cmd, capture_output=True,
                                text=True, check=True).stdout

            with open(log_file, "a") as f:
                f.write(f"--- Test {i} of 10 ---\n{out}\n")

            match = re.search(
                r"0\.\d+\.\d+\s+sec.*?\s+(\d+(?:\.\d+)?)\s+[M|G]bits/sec", out
            )
            if match:
                tputs.append(float(match.group(1)))

            print(f"Completed Test {i}/10")

        except Exception as e:
            sys.exit(f"Fatal Error: {e}")
            time.sleep(1)

    if tputs:
        stats_text = (
            "\n--- STATISTICAL RESULTS (Mbits/sec) ---\n"
            f"Avg: {statistics.mean(tputs):.2f} | "
            f"Min: {min(tputs):.2f} | "
            f"Max: {max(tputs):.2f} | "
            f"StdDev: {(statistics.stdev(tputs) if len(tputs)>1 else 0):.2f}\n"
        )
        print(stats_text)
        with open(log_file, "a") as f:
            f.write(stats_text)

if __name__ == "__main__":
    main()
```

Listing 1: Automated iperf client measurement script.

```

581 1 import subprocess
582 2
583 3
584 4 def main():
585 5     print("=== iperf Server Mode ===")
586 6     protocol = input("Protocol (tcp/udp): ").strip().
        lower()
587 7
588 8     cmd = ["iperf", "-s"]
589 9     if protocol == "udp":
590 10         cmd.append("-u")
591 11
592 12     print(f"\nGenerated iperf command: {' '.join(cmd)}")
593 13     print("Server listening... Press Ctrl+C to stop.\n")
594 14
595 15     try:
596 16         subprocess.run(cmd)
597 17     except KeyboardInterrupt:
598 18         print("\nServer execution interrupted by user.")
599 19     except FileNotFoundError:
600 20         print("\nError: iperf is not installed or not in
        the system PATH.")
601
602 21
603 22
604 23 if __name__ == "__main__":
605 24     main()

```

Listing 2: Automated iperf server measurement script.

B RAW TERMINAL OUTPUTS

This appendix contains the raw terminal logs generated during the experimental tests. These outputs serve as a direct validation of the networking tools' execution and provide a low-level snapshot of the system's behavior.

B.1 Purely Ethernet Scenario

```

1 Settings for enp3s0:
2   Supported ports: [ TP ]
3   Supported link modes:
4       10baseT/Half 10baseT/Full
5       100baseT/Half 100baseT/Full
6       1000baseT/Half 1000baseT/Full
7   Supported pause frame use: Symmetric
8   Supports auto-negotiation: Yes
9   Supported FEC modes: Not reported
10  Advertised link modes:  10baseT/Half 10baseT/Full
11                          100baseT/Half 100baseT/Full
12  Advertised pause frame use: Symmetric Receive-only
13  Advertised auto-negotiation: Yes
14  Advertised FEC modes: Not reported
15  Link partner advertised link modes:
16      10baseT/Half 10baseT/Full
17      100baseT/Half 100baseT/Full
18      1000baseT/Full
19  Link partner advertised pause frame use: No
20  Link partner advertised auto-negotiation: Yes
21  Link partner advertised FEC modes: Not reported
22  Speed: 100Mb/s
23  Duplex: Full
24  Auto-negotiation: on
25  Port: Twisted Pair
26  PHYAD: 1
27  Transceiver: internal
28  Link detected: yes

```

Listing 3: Kubuntu ethernet interface status

```

1 Settings for enx7cc2c63eb602:
2   Supported ports: [ TP      MII ]
3   Supported link modes:
4       10baseT/Half 10baseT/Full
5       100baseT/Half 100baseT/Full
6       1000baseT/Half 1000baseT/Full
7   Supported pause frame use: No
8   Supports auto-negotiation: Yes
9   Supported FEC modes: Not reported
10  Advertised link modes:
11      10baseT/Half 10baseT/Full
12      100baseT/Half 100baseT/Full
13      1000baseT/Full
14  Advertised pause frame use: No
15  Advertised auto-negotiation: Yes
16  Advertised FEC modes: Not reported
17  Link partner advertised link modes:
18      10baseT/Half 10baseT/Full
19      100baseT/Half 100baseT/Full
20  Link partner advertised pause frame use:
21      Symmetric Receive-only
22  Link partner advertised auto-negotiation: Yes
23  Link partner advertised FEC modes: Not reported
24  Speed: 100Mb/s
25  Duplex: Full
26  Auto-negotiation: on
27  Port: MII
28  PHYAD: 3
29  Transceiver: internal
30 netlink error: Operation not permitted
31   Current message level: 0x00000007 (7)
32   drv probe link
33
34  Link detected: yes

```

Listing 4: Arch ethernet interface status

B.2 Purely Wi-Fi Scenario

```
1 Connected to b4:7c:a6:d3:17:4f (on wlo1)
2   SSID: TIM-42281955
3   freq: 5200
4   RX: 1401011083 bytes (2159106 packets)
5   TX: 461508440 bytes (1321663 packets)
6   signal: -57 dBm
7   rx bitrate: 1201.0 MBit/s 80MHz HE-MCS 11 HE-NSS 2
8   HE-GI 0
9   tx bitrate: 1201.0 MBit/s 80MHz HE-MCS 11 HE-NSS 2
10  HE-GI 0
11  bss flags: short-preamble short-slot-time
    dtim period: 1
    beacon int: 100
```

Listing 5: Kubuntu wireless interface status

```
1 Connected to b4:7c:a6:d3:17:4f (on wlan0)
2   SSID: TIM-42281955
3   freq: 5200.0
4   RX: 3010110839 bytes (4959106 packets)
5   TX: 761508440 bytes (2821663 packets)
6   signal: -43 dBm
7   rx bitrate: 1921.5 MBit/s 160MHz EHT-MCS 9
8   EHT-NSS 2 EHT-GI 0
9   tx bitrate: 2593.5 MBit/s 160MHz EHT-MCS 12
10  EHT-NSS 2 EHT-GI 0
11  bss flags: short-preamble short-slot-time
    dtim period: 1
    beacon int: 100
```

Listing 6: Arch wireless interface status

B.3 Mixed Scenario

```
1 Settings for enp0s13f0u1u4:
2   Supported ports: [ TP MII ]
3   Supported link modes:
4     10baseT/Half 10baseT/Full
5     100baseT/Half 100baseT/Full
6     1000baseT/Half 1000baseT/Full
7   Supported pause frame use: No
8   Supports auto-negotiation: Yes
9   Supported FEC modes: Not reported
10  Advertised link modes:
11    10baseT/Half 10baseT/Full
12    100baseT/Half 100baseT/Full
13    1000baseT/Full
14  Advertised pause frame use: No
15  Advertised auto-negotiation: Yes
16  Advertised FEC modes: Not reported
17  Link partner advertised link modes:
18    10baseT/Half 10baseT/Full
19    100baseT/Half 100baseT/Full
20    1000baseT/Half 1000baseT/Full
21  Link partner advertised pause frame use: No
22  Link partner advertised auto-negotiation: Yes
23  Link partner advertised FEC modes: Not reported
24  Speed: 1000Mb/s
25  Duplex: Full
26  Auto-negotiation: on
27  Port: MII
28  PHYAD: 32
29  Transceiver: internal
30  Supports Wake-on: pumbg
31  Wake-on: g
32  Current message level: 0x00007fff (32767)
33  drv probe link timer ifdown ifup rx_err tx_err
34  tx_queued intr tx_done rx_status pktdata hw
    wol
    Link detected: yes
```

Listing 7: Arch ethernet interface status

```
1 Connected to b4:7c:a6:d3:17:4f (on wlo1)
2   SSID: TIM-42281955
3   freq: 5200
4   RX: 1401011083 bytes (2159106 packets)
5   TX: 461508440 bytes (1321663 packets)
6   signal: -57 dBm
7   rx bitrate: 1201.0 MBit/s 80MHz HE-MCS 11 HE-NSS 2
8   HE-GI 0
9   tx bitrate: 1201.0 MBit/s 80MHz HE-MCS 11 HE-NSS 2
10  HE-GI 0
11  bss flags: short-preamble short-slot-time
    dtim period: 1
    beacon int: 100
```

Listing 8: Kubuntu wireless interface status

C DETAILED SCENARIO CONFIGURATIONS

This appendix provides the comprehensive **technical specifications** and network configurations for all three experimental scenarios: Ethernet, Wireless, and Mixed.

Table 4: iperf payload parameters

Parameter	TCP	UDP
MTU Size (Standard)	1500 Bytes	1500 Bytes
Protocol Payload	1460 Bytes	1470 Bytes

C.1 Ethernet Scenario

Table 5: Testbed Configuration: Ethernet Scenario

Parameter	STA 1 - kubuntu	STA 2 - arch
IP Address	169.254.64.70	169.254.21.153
Nominal C. (RX)	360.0 Mbit/s	324.0 Mbit/s
Nominal C. (TX)	300.0 Mbit/s	324.0 Mbit/s
ETH adapter	Tiergrade Gigabit	TP-LINK Megabit
Cable	Cat.5e	

C.2 Wireless Scenario

Table 6: Testbed Configuration: Wireless Scenario

Parameter	STA 1 - kubuntu	STA 2 - arch
SSID	<i>TIM-42281955</i>	<i>TIM-42281955</i>
BSSID	b4:7c:a6:d3:17:4f	b4:7c:a6:d3:17:4f
IP Address	192.168.1.4/24	192.168.1.11/24
Default Gateway	192.168.1.1	192.168.1.1
Signal Strength	-57 dBm	-43 dBm
Frequency	5200.0	5200.0
Nominal C. (RX)	1201.0 Mbps	1921.5 Mbps
Nominal C. (TX)	1201.0 Mbps	2593.5 Mbps
Wireless Std.	802.11ax (Wi-Fi 6)	802.11be (Wi-Fi 7)
Router	TIM Hub+	

C.3 Mixed Scenario

Table 7: Testbed Configuration: Mixed Scenario

Parameter	STA 1 - kubuntu	STA 2 - arch
IP Address	192.168.1.4/24	192.168.1.11/24
Default Gateway	192.168.1.1	192.168.1.1
Nominal C. (RX)	1201.0 Mbps	1000.0 Mbps
Nominal C. (TX)	1201.0 Mbps	1000.0 Mbps
SSID	<i>TIM-42281955</i>	-
BSSID	b4:7c:a6:d3:17:4f	-
Signal Strength	-57 dBm	-
Frequency	5200.0	-
Wireless Std.	802.11ax (Wi-Fi 6)	-
Cable	-	Cat.6
Router	TIM Hub+	